The GenAI Engineer Handbook

A Practical Guide to Building, Deploying, and Scaling Generative AI Systems

Compiled Reference Edition

2026

Table of Contents

Preface

Generative AI engineering emerged as a distinct discipline in the early 2020s, sitting at the intersection of machine learning research, software engineering, and product design. Unlike traditional machine learning engineering, which centers on training models from scratch for narrow prediction tasks, GenAI engineering is largely about building systems *around* large pretrained models: retrieval pipelines, orchestration layers, evaluation harnesses, guardrails, and the infrastructure that lets these systems operate reliably in production.

This handbook is organized as a progressive curriculum. Early chapters cover the theoretical foundations a GenAI engineer needs — how large language models actually work, what a transformer is doing internally, and how tokens become predictions. Later chapters move into the practical craft: prompting, retrieval-augmented generation, fine-tuning, agentic systems, evaluation, and the operational discipline required to run these systems at scale. The final chapters address governance, security, and career development.

No single document can make anyone an expert. Treat this as a map of the territory — a reference to return to as you build, break, and rebuild real systems.

Chapter 1: Introduction to Generative AI Engineering

1.1 What Is a GenAI Engineer?

A GenAI engineer designs, builds, and operates systems that use generative models — most commonly large language models (LLMs), but increasingly also diffusion models, multimodal models, and speech models — to accomplish tasks that were previously impossible or required significant human effort. The role sits between three traditional disciplines:

- **Machine learning engineering**, which contributes an understanding of model architectures, training dynamics, and evaluation methodology.
- **Backend/platform engineering**, which contributes the discipline of building reliable, observable, and scalable services.

- **Product engineering**, which contributes an understanding of how to translate ambiguous user needs into a working system, and how to iterate based on real usage.

In practice, a GenAI engineer spends far less time training models than the job title might suggest. Most organizations do not train foundation models from scratch; they consume models via API or self-hosted open-weight checkpoints, and the engineering work is in how those models are integrated, constrained, evaluated, and monitored.

1.2 The Core Responsibilities

A typical GenAI engineering role includes some mixture of the following:

- Designing prompts and prompt templates that reliably elicit the desired model behavior.
- Building retrieval systems that ground model outputs in an organization's proprietary data.
- Fine-tuning or otherwise adapting models to a narrower task or style.
- Building agentic workflows where a model plans, calls tools, and iterates toward a goal.
- Constructing evaluation suites that measure quality, safety, and regression across model or prompt changes.
- Operating the production infrastructure: latency, cost, rate limits, caching, and failover.
- Implementing safety guardrails, content filters, and abuse-prevention systems.
- Collaborating with legal, security, and compliance teams on data handling and model risk.

1.3 How This Differs From Classical ML Engineering

Classical ML engineering typically follows a well-defined loop: collect labeled data, engineer features, train a model, evaluate against a held-out test set, and deploy. The unit of iteration is the model itself.

GenAI engineering replaces much of that loop with system design around a model that is already highly capable out of the box. The unit of iteration becomes the *system*: the prompt, the retrieved context, the tool definitions, the guardrails, and the orchestration logic. This has several consequences:

1. **Evaluation is harder.** There is rarely a single ground-truth label; outputs are open-ended text, and quality is often subjective or multi-dimensional (correctness, tone, safety, brevity).
2. **Iteration is faster but noisier.** You can change a prompt and redeploy in minutes, but the effect of that change may be inconsistent across inputs.
3. **Non-determinism is a first-class concern.** The same input can produce different outputs across calls, which changes how you think about testing, caching, and reproducibility.
4. **Cost and latency are architectural constraints**, not just afterthoughts, because every additional model call has a real dollar and millisecond cost.

1.4 The Modern GenAI Stack

A production GenAI system typically has several distinct layers:

Layer	Purpose	Example Components
Model layer	Provides the underlying generative capability	Hosted APIs, self-hosted open-weight models
Orchestration layer	Sequences calls, manages state, invokes tools	Agent frameworks, workflow engines
Retrieval layer	Grounds generation in external knowledge	Vector databases, search indexes
Data layer	Stores documents, embeddings, and conversation history	Object storage, relational databases
Evaluation layer	Measures quality and catches regressions	Automated test suites, human review pipelines
Observability layer	Tracks latency, cost, errors, and drift	Tracing systems, dashboards, alerting
Safety layer	Filters unsafe inputs and outputs	Moderation models, rule-based filters

Understanding how these layers interact — and where responsibility for a given behavior actually lives — is one of the most valuable mental models a GenAI engineer can develop.

Chapter 2: Foundations — How Large Language Models Work

2.1 From Text to Tokens

Language models do not operate on raw text; they operate on sequences of tokens, which are typically sub-word units produced by a tokenizer such as byte-pair encoding (BPE) or a variant like SentencePiece. A word like “generation” might be a single token, while a rarer word might be split into several sub-word pieces. Understanding tokenization matters practically because:

- Pricing for hosted APIs is almost always per-token, not per-word or per-character.
- Context window limits are expressed in tokens, so the same prompt can consume different amounts of budget depending on language and formatting.
- Certain failure modes (poor arithmetic, difficulty with rare words, sensitivity to whitespace) trace directly back to how tokenization fragments the input.

2.2 The Language Modeling Objective

At training time, a language model learns to predict the next token in a sequence, given all preceding tokens. This objective, applied over enormous corpora of text, forces the model to internalize a huge amount of world knowledge, linguistic structure, and reasoning patterns as a side effect of getting good at prediction. It is important to remember that this objective is

fundamentally about *plausibility*, not truth — a model is trained to produce the token that is statistically likely to follow, not the token that is factually correct. This is the root cause of hallucination, and no amount of prompt engineering fully eliminates it.

2.3 Pretraining, Instruction Tuning, and Alignment

Modern chat-oriented LLMs are typically produced through several stages:

1. **Pretraining** on a broad corpus of text (and increasingly code, and multimodal data) using the next-token prediction objective. This is by far the most computationally expensive stage and is where the bulk of the model's raw capability originates.
2. **Supervised fine-tuning (SFT)** on curated examples of desired input-output behavior, which teaches the model to follow instructions and adopt a helpful, conversational format.
3. **Preference-based alignment**, commonly using techniques descended from reinforcement learning from human feedback (RLHF) or more recent direct-preference methods, which further shapes the model's outputs toward being helpful, honest, and harmless according to human or model-generated preference judgments.

Each stage changes the model's behavior in ways relevant to engineering. A base (pretrained-only) model will often continue text rather than answer a question; an instruction-tuned model is shaped for conversational, task-following behavior; an aligned model has additional shaping toward refusing certain requests and adopting particular tones.

2.4 Context Windows and Their Practical Limits

The context window is the maximum number of tokens a model can attend to in a single call, encompassing the system prompt, conversation history, retrieved documents, and the model's own output. While context windows have grown dramatically — from a few thousand tokens in early models to several hundred thousand in current frontier models — a larger context window does not mean uniformly good use of that context. Empirically, models tend to attend more reliably to information near the beginning and end of a long context than to material buried in the middle, an effect often described informally as a "lost in the middle" phenomenon. Engineers should treat a large context window as capacity to be used deliberately, not as a substitute for good retrieval and prompt design.

2.5 Sampling and Decoding

At inference time, a model produces a probability distribution over the next token, and a decoding strategy selects which token to actually emit. Common strategies include:

- **Greedy decoding**: always pick the highest-probability token. Deterministic but can produce repetitive, low-diversity output.
- **Temperature sampling**: scale the probability distribution before sampling; higher temperature increases randomness and diversity, lower temperature makes output more deterministic and focused.

- **Top-p (nucleus) sampling**: sample only from the smallest set of tokens whose cumulative probability exceeds a threshold p, which avoids sampling from the long tail of implausible tokens while preserving diversity.
- **Top-k sampling**: restrict sampling to the k most probable tokens.

For tasks requiring factual precision (code generation, data extraction, structured output) lower temperature is generally preferred. For creative or brainstorming tasks, higher temperature can produce more varied and interesting output.

Chapter 3: Transformer Architecture Deep Dive

3.1 Why the Transformer Won

Before the transformer architecture, sequence modeling relied heavily on recurrent neural networks (RNNs) and their variants (LSTMs, GRUs), which process tokens one at a time in sequence. This made training slow, since it could not be easily parallelized, and made it difficult for the model to retain information over long distances in a sequence. The transformer architecture, introduced in 2017, replaced recurrence with a mechanism called self-attention, which allows every token to directly attend to every other token in a sequence in parallel. This unlocked massive parallelization during training and dramatically improved the modeling of long-range dependencies, and it remains the dominant architecture behind essentially all modern large language models.

3.2 Self-Attention, Intuitively

Self-attention allows each token's representation to be updated based on a weighted combination of all other tokens' representations, where the weights are learned and depend on the content of the tokens themselves. Concretely, each token is projected into three vectors: a query, a key, and a value. The similarity between a token's query and every other token's key determines how much attention that token pays to each other token; those attention weights are then used to combine the value vectors into an updated representation. This mechanism is what allows a model to, for example, resolve that "it" in a sentence refers to a particular noun mentioned several words earlier.

3.3 Multi-Head Attention

Rather than computing a single attention pattern, transformers compute several attention "heads" in parallel, each with its own learned query, key, and value projections. Different heads often specialize in different kinds of relationships — some attend to syntactic structure, others to coreference, others to longer-range topical relationships. The outputs of all heads are concatenated and projected back into the model's working dimensionality.

3.4 Positional Information

Because self-attention treats a sequence as a set rather than an ordered list, transformers need an explicit mechanism to encode token position. Early transformers used fixed sinusoidal positional encodings added to token embeddings; modern large language models more

commonly use relative positional encoding schemes such as Rotary Position Embeddings (RoPE), which encode relative distance between tokens directly into the attention computation and generalize better to sequence lengths not seen during training.

3.5 Feed-Forward Layers and Residual Connections

Each transformer block interleaves self-attention with a position-wise feed-forward network (typically a two-layer multilayer perceptron with a nonlinearity), and both sub-layers are wrapped in residual connections and layer normalization. The feed-forward layers are where a large fraction of the model's parameters live, and are widely believed to function as a kind of key-value memory storing learned facts and patterns. The residual connections are critical for enabling stable training of very deep networks — without them, gradients would struggle to propagate through dozens of stacked layers.

3.6 Scaling Laws

Empirical research has repeatedly shown that model performance improves predictably as a power-law function of three quantities: model size (parameter count), dataset size (tokens seen during training), and compute budget. These relationships, often called scaling laws, have guided the industry's investment in ever-larger training runs, though more recent work has emphasized that data quality and quantity must scale in proportion to model size for compute to be used efficiently — a finding that shifted the field's emphasis from "bigger models" toward "better-balanced compute-optimal training."

3.7 Mixture-of-Experts Architectures

An increasingly common architectural variant is the mixture-of-experts (MoE) transformer, in which the feed-forward layer is replaced with a set of multiple "expert" sub-networks, and a learned routing mechanism selects a small subset of experts to activate for each token. This allows the total parameter count of a model to grow substantially while keeping the computational cost of each forward pass much lower than a dense model of equivalent size, since only a fraction of parameters are active for any given token. MoE architectures introduce their own engineering challenges, including load-balancing across experts and increased memory requirements for hosting the full parameter set even though only part of it is used per token.

Chapter 4: Prompt Engineering Fundamentals

4.1 What Prompt Engineering Actually Is

Prompt engineering is the practice of designing the input given to a language model — including instructions, examples, context, and formatting — to reliably elicit a desired output. It is often underestimated as a skill because a well-crafted prompt can look deceptively simple; the difficulty lies in the iterative process of discovering what a particular model responds to, and in building prompts that remain robust across a wide distribution of real user inputs, not just the handful of examples used during development.

4.2 Anatomy of a Well-Structured Prompt

Most effective prompts combine several distinct components:

- **Role or persona framing**, which sets the frame the model should adopt (e.g., "You are a meticulous financial analyst").
- **Task instructions**, stated clearly and unambiguously, ideally as a numbered or step-by-step description of what to do.
- **Context or reference material**, such as retrieved documents, prior conversation, or user-supplied data the model should ground its answer in.
- **Output format specification**, describing exactly how the response should be structured — plain text, JSON, markdown, a specific schema, a length constraint.
- **Examples (few-shot demonstrations)**, showing input-output pairs that illustrate the desired behavior, especially useful for tasks with a specific style or format that is hard to describe in words alone.
- **Constraints and negative instructions**, clarifying what the model should avoid doing.

4.3 Zero-Shot, Few-Shot, and In-Context Learning

A zero-shot prompt gives the model only an instruction, with no examples. A few-shot prompt includes a small number of example input-output pairs before the actual query, leveraging the model's in-context learning ability — its capacity to infer a task pattern from examples provided directly in the prompt, without any weight updates. Few-shot prompting is particularly effective for tasks with idiosyncratic formatting requirements, classification tasks with a fixed label set, or tasks where the desired style is easier to demonstrate than describe. The tradeoff is token cost: every example consumes context budget on every single call.

4.4 Being Explicit and Unambiguous

Language models are highly literal in ways that can surprise engineers used to communicating with other humans, who fill in unstated assumptions automatically. Vague instructions such as "make it better" or "summarize this" leave enormous room for interpretation, and different models — or the same model on different days, if the underlying deployment changes — may interpret them differently. Effective prompts specify:

- The desired length (a word or sentence count, not just "brief" or "detailed").
- The audience and tone (a technical audience versus a general one).
- What "good" looks like, ideally with a concrete example or a rubric.
- Edge case handling (what should happen if the input is empty, contradictory, or out of scope).

4.5 System Prompts vs. User Prompts

Most chat-based model APIs distinguish between a system prompt (persistent instructions that frame the model's behavior for the entire conversation) and user prompts (the individual turns of the conversation). System prompts are the right place for persona, tone, output-format rules, and safety constraints that should apply consistently; user prompts carry the specific request or

query for a given turn. Engineers should avoid overloading the system prompt with content that changes per-request, since many caching and optimization strategies treat the system prompt as a stable prefix.

4.6 Common Prompting Failure Modes

- **Instruction dilution**: burying the most important instruction in the middle of a long prompt, where it receives less attention than instructions near the start or end.
- **Contradictory instructions**: asking for both extreme brevity and comprehensive detail, which forces the model to arbitrate between conflicting goals unpredictably.
- **Format drift**: not specifying an output format precisely enough, leading to inconsistent structure across calls that breaks downstream parsing.
- **Over-reliance on a single prompt version**: shipping a prompt that worked well on a handful of manually tested examples without validating it against a broader, more representative test set.

Chapter 5: Advanced Prompting Techniques

5.1 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting instructs a model to reason through a problem step by step before producing a final answer, rather than jumping directly to a conclusion. This technique reliably improves performance on tasks requiring multi-step reasoning, arithmetic, or logical deduction, because it gives the model additional token-by-token "working space" in which to build up intermediate conclusions, and each generated token can be conditioned on in generating the next. A simple instruction such as "think through this step by step before giving your final answer" is often sufficient to elicit this behavior in capable models, though explicitly structuring the output (e.g., a "Reasoning:" section followed by an "Answer:" section) makes the result easier to parse programmatically.

5.2 Self-Consistency

Self-consistency extends chain-of-thought by sampling multiple independent reasoning paths for the same problem (using a nonzero temperature) and then selecting the most common final answer among them, on the premise that correct reasoning paths are more likely to converge on the same answer than incorrect ones. This trades additional inference cost — multiple model calls per query — for improved reliability, and is most valuable for high-stakes or difficult reasoning tasks where the cost of an error outweighs the cost of extra compute.

5.3 Reasoning Models and Extended Thinking

A newer class of models is trained specifically to perform extended internal reasoning before producing a final response, often using reinforcement learning against verifiable reward signals (such as whether a math problem's answer is correct, or whether generated code passes a test suite). These models can be prompted more simply than earlier chain-of-thought techniques required, since the extended reasoning behavior is built into the model itself rather than elicited purely through prompt phrasing. Engineers working with such models should

understand that the reasoning trace may be a separate, sometimes hidden or summarized channel of output distinct from the final answer, with its own token costs and latency implications.

5.4 Structured Output and Function-Calling-Style Prompting

Many production use cases require output that a downstream system can parse reliably, rather than free-form prose. Several complementary strategies help here:

- **Explicit schema specification**, describing the exact fields, types, and structure expected, ideally paired with a formal schema (such as JSON Schema) that the model is asked to conform to.
- **Constrained decoding / grammar-based generation**, a lower-level technique supported by some inference stacks that restricts which tokens can be sampled at each step so the output is guaranteed to match a given grammar, eliminating the class of errors where a model produces almost-valid but not quite parseable output.
- **Native structured-output or function-calling APIs**, offered by most major model providers, which handle schema conformance at the API level rather than relying purely on prompt instructions.

5.5 Prompt Chaining and Decomposition

Complex tasks are often better handled by decomposing them into a sequence of smaller, more constrained prompts rather than attempting to solve the entire task in a single call. For example, a document-summarization-and-translation task might be split into a summarization step followed by a separate translation step, each with a narrowly scoped prompt, rather than one prompt attempting both simultaneously. This decomposition improves reliability (each step is individually easier to get right and easier to evaluate), but increases latency and cost due to multiple sequential model calls, so the decision to chain versus consolidate prompts is a real engineering tradeoff.

5.6 Adversarial Robustness of Prompts

Prompts deployed in production are exposed to adversarial or simply unusual user input, and engineers should design and test for this. Relevant failure modes include prompt injection (where user-supplied content, especially retrieved documents, contains text designed to override the system's instructions), jailbreak attempts (crafted inputs designed to bypass safety behavior), and simple edge cases like empty input, extremely long input, or input in an unexpected language. Defensive techniques include clearly delimiting untrusted content within the prompt, instructing the model explicitly to treat retrieved or user-supplied content as data rather than instructions, and validating model output before it is used downstream (e.g., before executing a tool call the model requested).

Chapter 6: Embeddings and Vector Representations

6.1 What an Embedding Is

An embedding is a dense vector representation of a piece of content — a word, sentence, paragraph, image, or other data — produced by a model trained so that semantically similar inputs are mapped to nearby points in the vector space. Unlike sparse representations such as one-hot encodings or bag-of-words, embeddings capture semantic relationships: the vectors for "physician" and "doctor" will be close together, even though the words share no characters, because the embedding model learned that they are used in similar contexts.

6.2 How Embedding Models Are Trained

Modern text embedding models are typically trained using contrastive learning objectives, where the model learns to pull together the vector representations of semantically related pairs (such as a question and its correct answer, or two paraphrased sentences) while pushing apart the representations of unrelated pairs. Some embedding models are derived from general-purpose language models via additional fine-tuning specifically for retrieval tasks; others are trained from scratch on large corpora of paired or weakly-paired text.

6.3 Similarity Metrics

Once content is embedded, similarity between two vectors is typically measured using one of a small number of standard metrics:

- **Cosine similarity**, which measures the angle between two vectors and is invariant to their magnitude — the most commonly used metric for text embeddings.
- **Dot product**, which is related to cosine similarity but also incorporates vector magnitude; some embedding models are specifically trained to be compared via dot product rather than cosine similarity.
- **Euclidean (L2) distance**, which measures straight-line distance between vectors and is more common in image and general-purpose vector search than in text retrieval.

Choosing the correct metric matters: using the wrong similarity function for a given embedding model can silently degrade retrieval quality, since models are trained and calibrated against a specific metric.

6.4 Dimensionality and Its Tradeoffs

Embedding dimensionality — commonly ranging from a few hundred to a few thousand dimensions — affects both representational capacity and system cost. Higher-dimensional embeddings can capture more nuanced semantic distinctions but require more storage and more compute for similarity search, and increase the cost of any downstream indexing structure. Several modern embedding models support Matryoshka representation learning, which trains a single embedding such that its leading sub-vectors (e.g., the first 256 dimensions of a 1024-dimensional embedding) are themselves valid, high-quality embeddings at lower dimensionality. This allows a system to trade off retrieval quality against storage and compute cost dynamically, without retraining or re-embedding the underlying data.

6.5 Chunking Strategy and Its Impact on Embedding Quality

Before content can be embedded, it typically must be split into chunks of manageable size, since embedding models have a maximum input length and because embedding an entire long document into a single vector tends to dilute its semantic specificity. Chunking strategy is one of the highest-leverage, most underrated decisions in a retrieval system:

- **Fixed-size chunking** splits text into chunks of a set token or character length, optionally with overlap between consecutive chunks to avoid losing context at chunk boundaries. Simple to implement but can split sentences or ideas awkwardly.
- **Semantic chunking** attempts to split text at natural topic or section boundaries, using heuristics such as heading structure, paragraph breaks, or even a model-based estimate of semantic discontinuity between adjacent sentences.
- **Recursive chunking** applies a hierarchy of separators (attempting to split on section breaks first, then paragraphs, then sentences) until chunks fall within a target size range.
- **Document-aware chunking** respects the structure of specific document types — treating a table as an atomic unit, keeping code blocks intact, or preserving the association between a heading and the content beneath it.

Poor chunking can silently degrade an entire retrieval pipeline in ways that are difficult to diagnose downstream, since the failure surfaces as "the model gave a wrong answer" rather than as an obvious chunking bug.

Chapter 7: Vector Databases

7.1 Why Specialized Vector Storage Exists

Searching for the nearest neighbors of a query vector among millions or billions of stored embeddings using brute-force comparison is computationally expensive at scale. Vector databases and vector search libraries exist to make this operation fast, typically by building an index structure that allows approximate nearest neighbor (ANN) search — trading a small amount of recall accuracy for orders-of-magnitude improvements in search speed.

7.2 Common Indexing Algorithms

- **HNSW (Hierarchical Navigable Small World)**, a graph-based indexing structure that builds multiple layers of proximity graphs, enabling fast approximate search by navigating from coarse to fine layers. HNSW is widely used because it offers a strong balance of speed, recall, and support for incremental updates.
- **IVF (Inverted File Index)**, which partitions the vector space into clusters (via a method like k-means) and, at query time, searches only within the clusters nearest to the query vector, rather than the entire dataset.
- **Product quantization**, a compression technique often combined with IVF, which compresses vectors into compact codes to reduce memory footprint at some cost to precision.

- **Exact / brute-force search**, still practical for smaller datasets (tens of thousands of vectors) where the simplicity and perfect recall outweigh the performance cost.

7.3 Choosing a Vector Database

Relevant considerations when selecting a vector database for a production system include: whether it needs to be self-hosted or is acceptable as a managed service; whether it needs to support hybrid search combining vector similarity with traditional keyword or metadata filtering; how it handles incremental updates and deletions; its support for metadata filtering at query time (essential for multi-tenant systems where results must be scoped to a particular user or document set); and its operational maturity, including backup, replication, and monitoring support.

7.4 Hybrid Search

Pure vector similarity search can underperform on queries that depend heavily on exact terms — product codes, proper nouns, acronyms, or numeric identifiers — because semantic embeddings can treat superficially different but semantically similar terms as close, while missing the importance of an exact literal match. Hybrid search combines vector similarity with traditional lexical search techniques such as BM25, then merges the two result sets using a fusion method such as reciprocal rank fusion. In practice, hybrid search frequently outperforms either method alone and has become close to a default expectation for production-grade retrieval systems.

7.5 Reranking

A common and effective pattern is a two-stage retrieval pipeline: an initial fast retrieval stage (vector or hybrid search) returns a relatively large candidate set (e.g., the top 50-100 results), and a second, more computationally expensive reranking model then scores and reorders those candidates using a deeper, more accurate relevance model — often a cross-encoder that jointly processes the query and each candidate document rather than comparing precomputed independent embeddings. This two-stage approach balances the speed of approximate vector search with the accuracy of a heavier relevance model, without paying the cost of running the heavy model over the entire corpus.

Chapter 8: Retrieval-Augmented Generation (RAG)

8.1 The Core Idea

Retrieval-augmented generation combines a retrieval system with a generative model: rather than relying solely on knowledge encoded in a model's weights during training, a RAG system retrieves relevant documents or passages from an external knowledge source at query time and includes them in the model's context, grounding the generated response in that retrieved material. RAG addresses two fundamental limitations of relying on a model's parametric knowledge alone: the model's knowledge is frozen at its training cutoff and cannot reflect new or private information, and the model's parametric recall of specific facts is unreliable and prone to hallucination.

8.2 A Basic RAG Pipeline

A minimal RAG pipeline consists of an offline indexing phase and an online query phase. In the indexing phase, source documents are collected, cleaned, chunked, embedded, and stored in a vector index along with relevant metadata. In the query phase, an incoming user query is embedded using the same embedding model, the vector index is searched for the most relevant chunks, and those chunks are inserted into a prompt template along with the original query, which is then sent to the generative model to produce a grounded answer.

8.3 Where Basic RAG Falls Short

Naive RAG implementations frequently underperform in production for reasons that are not obvious from the basic architecture:

- **Retrieval failures are silent.** If the relevant document was never retrieved, the model will either hallucinate an answer or, if instructed carefully, decline to answer — but the underlying cause (bad chunking, bad embedding, bad query) is not visible from the final output alone.
- **Queries and documents are often phrased very differently**, a mismatch sometimes called the vocabulary gap, which can cause otherwise relevant documents to score poorly against a literal query embedding.
- **Multi-hop questions**, which require combining information from multiple documents, are poorly served by a single retrieval pass that looks for chunks similar to the original query alone.
- **Context window misuse**: simply stuffing more retrieved chunks into the prompt does not reliably improve answer quality and can dilute the model's attention or exceed practical context limits.

8.4 Query Transformation Techniques

Several techniques address the vocabulary gap and improve retrieval quality by transforming the query before searching:

- **Query rewriting**, where a model reformulates the user's raw query into a more effective search query, useful for conversational contexts where a query like "what about last year?" depends on prior turns for meaning.
- **Hypothetical Document Embeddings (HyDE)**, where a model first generates a hypothetical answer to the query, and that hypothetical answer — rather than the query itself — is embedded and used for retrieval, on the premise that a plausible answer is often closer in embedding space to the real answer than the question is.
- **Query decomposition**, where a complex multi-part question is broken into several simpler sub-queries, each retrieved independently, with results combined before generation.
- **Multi-query expansion**, generating several paraphrased versions of the original query and retrieving for each, then merging and deduplicating the results, which improves recall against the vocabulary gap.

8.5 Grounding and Citation

For many production use cases — especially those with legal, medical, or financial implications — it is not enough for an answer to be correct; the system must also be able to show its work by citing which retrieved source(s) support each claim. This is typically implemented by tagging each retrieved chunk with a source identifier, instructing the model to cite sources inline using that identifier, and then post-processing the output to render those citations as links or footnotes. Citation also serves as a practical guardrail: a model instructed to cite its sources for every claim is, in practice, somewhat less likely to fabricate ungrounded claims, since the citation requirement nudges it toward relying on the provided context.

8.6 Evaluating RAG Systems

RAG evaluation should assess both the retrieval stage and the generation stage, since a failure in either can produce a bad final answer. Common metrics include:

- **Retrieval metrics**: precision and recall of retrieved chunks against a labeled set of relevant documents, and ranking-quality metrics such as mean reciprocal rank or normalized discounted cumulative gain (NDCG).
- **Faithfulness / groundedness**: whether the generated answer is actually supported by the retrieved context, as opposed to relying on the model's parametric knowledge or fabricating information.
- **Answer relevance**: whether the generated answer actually addresses the user's original question, independent of whether it is grounded.
- **Context precision and recall**: whether the retrieved chunks are relevant to the query (precision) and whether all necessary information was retrieved (recall).

Chapter 9: Advanced RAG Architectures

9.1 Agentic RAG

Agentic RAG extends the basic retrieve-then-generate pattern by giving the model agency over the retrieval process itself: rather than performing a single fixed retrieval step, the model can decide to issue additional queries, choose between multiple retrieval sources, or determine that no further retrieval is necessary, iterating until it has gathered sufficient information to answer. This is typically implemented by exposing retrieval as a tool the model can call, within an agentic loop (see Chapter 12), rather than as a hardcoded pipeline stage.

9.2 Graph-Based RAG

Graph-based RAG systems build a knowledge graph — entities and the relationships between them — from the source corpus, in addition to or instead of a flat vector index. This allows the retrieval system to answer questions that depend on relationships between entities (e.g., "which suppliers does Company X share with Company Y?") that are difficult to answer from independently embedded document chunks. Graph construction typically uses an LLM to extract entities and relationships from source documents, which are then stored in a graph

database and can be queried through graph traversal, vector similarity over graph node embeddings, or a hybrid of both.

9.3 Hierarchical and Summary-Based Retrieval

For very large corpora, some architectures build a hierarchy of summaries — for example, summarizing clusters of related documents, then summarizing clusters of those summaries — and allow retrieval to operate at multiple levels of the hierarchy depending on the specificity of the query. A broad question might be answered well from a high-level summary node, while a narrow, specific question requires descending to the original source chunks. This approach, sometimes implemented via recursive clustering and summarization, can improve performance on questions that require a broad synthesis across many documents, which naive flat retrieval handles poorly since it tends to surface many individually-relevant but collectively redundant chunks.

9.4 Long-Context Models and the Future of RAG

As model context windows have grown, some practitioners have questioned whether RAG remains necessary when a model can simply ingest an entire corpus directly. In practice, RAG and long-context capability are complementary rather than substitutes: retrieval remains valuable for controlling cost (processing fewer tokens per call), for enabling access to corpora far larger than any context window (enterprise document repositories, for instance), for improving the reliability of attention over relevant information (models still perform less reliably as relevant information is diluted among large volumes of irrelevant context), and for providing clean citation and access-control boundaries. The most robust production architectures typically use retrieval to narrow a large corpus down to a manageable, highly relevant set of documents, then rely on a reasonably large context window to include that narrowed set with generous margin.

9.5 Multi-Modal Retrieval

Increasingly, source corpora include images, tables, charts, and other non-text content, and multi-modal RAG systems must handle retrieval across these modalities. Approaches include using multi-modal embedding models that map text and images into a shared vector space, extracting and separately embedding textual descriptions of visual content (e.g., using a vision-capable model to caption charts or tables), and preserving layout-aware document representations that keep an image and its surrounding textual context linked during chunking and retrieval.

Chapter 10: Fine-Tuning Large Language Models

10.1 When Fine-Tuning Is (and Isn't) the Right Tool

Fine-tuning updates a model's weights on a task-specific dataset, and is often reached for prematurely. Before fine-tuning, it is worth exhausting prompt engineering and RAG approaches, since fine-tuning introduces real costs: dataset curation, training infrastructure, ongoing maintenance as base models improve, and evaluation overhead. Fine-tuning tends to

be the right tool when the desired behavior is difficult to specify through instructions alone — a highly specific tone or style, a structured output format the model consistently gets subtly wrong, domain-specific terminology or reasoning patterns not well represented in the base model's training data, or a need to reduce latency and cost by distilling a larger model's behavior into a smaller, cheaper fine-tuned model.

10.2 Full Fine-Tuning

Full fine-tuning updates all of a model's parameters using gradient descent against a task-specific loss, typically a supervised next-token prediction loss over curated input-output examples. This offers the greatest flexibility and can produce the strongest task-specific performance, but requires substantial compute — memory for the model weights, optimizer states, and gradients, which for large models can require many high-end GPUs — and carries a meaningful risk of catastrophic forgetting, where the model's general capabilities degrade as it over-specializes to the fine-tuning dataset.

10.3 Dataset Curation for Fine-Tuning

The quality of a fine-tuning dataset matters more than its size in most practical scenarios. Effective datasets typically emphasize:

- **Diversity of examples** covering the range of inputs the model will see in production, including edge cases and unusual phrasings.
- **Consistency of format and style**, since inconsistent examples teach the model inconsistent behavior.
- **Correctness**, since a fine-tuning dataset with a meaningful fraction of low-quality or incorrect examples will teach the model those errors.
- **Appropriate scale** — modern fine-tuning, particularly parameter-efficient approaches, can produce meaningful improvements from as few as a few hundred to a few thousand high-quality examples, rather than requiring the massive datasets associated with pretraining.

10.4 Data Synthesis and Distillation

Constructing a high-quality fine-tuning dataset manually is expensive, and it has become common to use a more capable model to generate or augment training examples for fine-tuning a smaller model — a process often called distillation. This might involve generating synthetic input-output pairs directly, using a strong model to rewrite or improve existing examples, or using a strong model to filter a larger, noisier dataset down to its highest-quality subset. Care must be taken to validate synthetic data quality, since errors or biases in the teacher model's outputs will propagate into the fine-tuned student model.

Chapter 11: Parameter-Efficient Fine-Tuning

11.1 The Case for Parameter Efficiency

Full fine-tuning of large models is often impractical outside of well-resourced organizations, both because of the compute required and because storing a separate full copy of a model's weights for every fine-tuned variant is expensive. Parameter-efficient fine-tuning (PEFT) methods update only a small subset of parameters — often less than one percent of the total — while achieving performance close to full fine-tuning on many tasks, dramatically reducing compute and storage requirements.

11.2 LoRA (Low-Rank Adaptation)

LoRA is currently the most widely used PEFT technique. Rather than updating a weight matrix directly, LoRA freezes the original weights and injects a pair of small, low-rank matrices whose product approximates the desired weight update. Only these low-rank matrices are trained, which typically represent a tiny fraction of the original parameter count. At inference time, the low-rank update can either be merged back into the original weights (producing a standalone fine-tuned model with no additional latency) or kept separate and applied dynamically, which allows a single base model deployment to serve many different LoRA adapters — one per customer or task, for example — with low incremental memory cost per adapter.

11.3 QLoRA and Quantization

QLoRA combines LoRA with quantization of the frozen base model weights, typically to 4-bit precision, dramatically reducing the memory footprint required to fine-tune large models. This makes it possible to fine-tune models with tens of billions of parameters on a single consumer or prosumer-grade GPU that would otherwise require a multi-GPU server for full-precision fine-tuning. Quantization introduces a small amount of numerical error, but QLoRA's design — including techniques like double quantization and paged optimizers — has been shown empirically to achieve performance close to full-precision fine-tuning on many benchmarks.

11.4 Choosing LoRA Hyperparameters

Key hyperparameters for LoRA-based fine-tuning include the rank (the dimensionality of the low-rank matrices, which controls the adapter's capacity — higher rank allows the adapter to represent more complex updates but increases parameter count and risk of overfitting), the alpha scaling factor (which controls the magnitude of the adapter's contribution relative to the frozen base weights), and which layers to apply the adapter to (commonly the attention projection matrices, though applying LoRA to feed-forward layers as well can improve performance for some tasks at additional parameter cost). As with most machine learning hyperparameters, appropriate values are best determined empirically for a given task and dataset rather than assumed from general guidance.

11.5 Merging, Serving, and Managing Adapters

Organizations that fine-tune many task- or customer-specific adapters against a shared base model need infrastructure to manage this effectively: a registry tracking which adapter

corresponds to which task or customer, versioning as adapters are retrained, and — for high-throughput serving — inference infrastructure capable of efficiently batching requests across multiple different adapters sharing the same base model weights, a capability supported by several modern inference-serving frameworks designed specifically for multi-adapter serving.

Chapter 12: Agentic AI Systems

12.1 What Makes a System "Agentic"

An agentic system is one in which a model does not simply produce a single response to a single input, but instead operates in a loop: observing its environment (including the results of its own prior actions), deciding on a next action, executing that action (often by calling an external tool), and repeating until it determines the task is complete or a stopping condition is reached. This stands in contrast to a simple single-turn generation, and introduces genuinely new engineering challenges around state management, error recovery, and bounding the agent's behavior.

12.2 The ReAct Pattern

One of the most influential and widely implemented agentic patterns is ReAct (Reasoning and Acting), in which the model alternates between generating a reasoning trace about what to do next and taking an action (typically a tool call), observing the result of that action, and incorporating that observation into its next reasoning step. This interleaving of explicit reasoning with action allows the model to adapt its plan based on real intermediate results, rather than committing to a rigid predetermined sequence of steps.

12.3 Planning Strategies

More sophisticated agentic architectures separate planning from execution: a planning step produces an explicit multi-step plan before any actions are taken, which can then be executed sequentially, with the option to re-plan if an action's outcome deviates from what was expected. This can improve reliability on complex, multi-step tasks compared to a purely reactive step-by-step approach, at the cost of additional latency and the risk that an early, poorly-formed plan constrains the agent even after new information suggests a different approach would be better.

12.4 State and Memory Management

Agentic systems that operate over many steps or long time horizons need a strategy for managing state — the accumulated context of what has happened so far — since simply appending every observation to an ever-growing prompt eventually exceeds context limits and dilutes attention. Common strategies include periodic summarization of older conversation or action history, maintaining a structured working memory of key facts distinct from the raw interaction log, and retrieval-based memory, where past interactions are stored externally and relevant portions are retrieved back into context only when needed.

12.5 Bounding Agent Behavior

Unbounded agentic loops carry real operational risk: an agent might loop indefinitely without making progress, take actions with real-world side effects that are difficult to reverse, or consume unbounded cost through repeated model calls. Production agentic systems should implement explicit bounds and safeguards: a maximum number of steps or a maximum time budget, a maximum cost budget per task, human-in-the-loop approval gates before executing high-consequence actions (financial transactions, sending communications, deleting data), and monitoring that can detect and halt loops that are not converging.

Chapter 13: Tool Use and Function Calling

13.1 The Function-Calling Paradigm

Function calling (also called tool use) allows a model to indicate that, rather than responding directly to a user, it wants to invoke a specific external function with specific arguments — for example, looking up a database record, calling a weather API, or executing a calculation. The model itself does not execute the function; it produces a structured description of the function call it wants made (typically the function name and arguments as JSON), which the calling application is responsible for actually executing, with the result then returned to the model as an additional message so it can incorporate that information into its next response.

13.2 Designing Good Tool Definitions

The quality of an agent's tool use is heavily influenced by how well the tools themselves are defined and documented for the model. Effective tool definitions include a clear, specific description of what the tool does and when it should be used (as opposed to when a different tool, or no tool, is more appropriate), a well-specified parameter schema with descriptive parameter names and types, and — critically — clear documentation of what the tool returns, including how errors are represented, so the model can reason about and handle failure cases appropriately. Tools with overlapping or ambiguous purposes are a common source of agent errors, where the model selects the wrong tool for a task because their descriptions were not sufficiently distinguishing.

13.3 Handling Tool Errors and Retries

Real-world tool calls fail: APIs time out, return malformed data, or reject invalid arguments. A robust agentic system needs to return errors to the model in a way that allows it to recover — clearly indicating that an error occurred, what kind of error it was, and, where possible, what the model might try instead — rather than simply crashing or returning an opaque failure that leaves the model unable to adapt. Application-level retry logic with backoff is appropriate for transient failures (network timeouts), while errors resulting from malformed model-generated arguments are usually better surfaced back to the model so it can correct its own request.

13.4 The Model Context Protocol and Tool Standardization

As agentic systems have proliferated, standardized protocols have emerged for exposing tools and data sources to models in a consistent way, decoupling tool implementation from any single model provider or application. Such protocols typically define a standard way for a "server" (exposing a set of tools, resources, or prompts) to communicate with a "client" (the application orchestrating a model's use of those tools), allowing tool providers to build once and have their integration work across many different agentic applications, and allowing application developers to connect to a growing ecosystem of pre-built tool integrations rather than building each one from scratch.

13.5 Security Implications of Tool Use

Granting a model the ability to invoke real-world actions introduces a security surface that does not exist in a pure text-generation system. Relevant risks include prompt injection leading to unauthorized tool calls (especially when a tool's output, or retrieved content, can itself contain text that influences the model's subsequent behavior), overly broad tool permissions (a tool that can delete any record when the task only requires reading one), and insufficient validation of tool arguments before execution. Defense in depth is the appropriate posture: scope tool permissions as narrowly as possible, validate and sanitize arguments at the application layer regardless of what the model produced, and apply the principle of least privilege to any credentials the tool-execution layer holds.

Chapter 14: Multi-Agent Orchestration

14.1 Why Use Multiple Agents

Some tasks are better decomposed across multiple specialized agents rather than handled by a single generalist agent attempting everything. Reasons to adopt a multi-agent architecture include separation of concerns (a research agent, a writing agent, and a fact-checking agent each have a narrower, more evaluable responsibility than one agent doing all three), the ability to give different agents different tools, permissions, or even different underlying models suited to their sub-task, and improved robustness through specialization, since narrowly-scoped agents are generally easier to prompt reliably than a single agent juggling many responsibilities at once.

14.2 Orchestration Patterns

Common multi-agent orchestration patterns include a supervisor/orchestrator pattern, where a coordinating agent decomposes a task and delegates sub-tasks to specialized worker agents, then synthesizes their outputs into a final result; a pipeline pattern, where agents operate in a fixed sequence, each consuming the previous agent's output; and a debate or critique pattern, where multiple agents (or multiple instances of the same agent) independently produce candidate solutions or critique each other's work, on the premise that this can surface errors a single pass would miss.

14.3 Communication and Shared State

Multi-agent systems need a well-defined mechanism for agents to share information — whether through a shared memory or blackboard that all agents can read and write, structured messages passed explicitly between agents, or a coordinating orchestrator that mediates all communication. Poorly designed communication is a common source of multi-agent system failures: information one agent generates may not be available in a form another agent can use, or agents may duplicate work because neither is aware of what the other has already done.

14.4 The Cost and Complexity Tradeoff

Multi-agent architectures multiply the operational challenges of a single agent — cost, latency, and debugging complexity all increase with the number of agents and the number of inter-agent communication steps. Before adopting a multi-agent architecture, it is worth validating that a well-engineered single-agent system with good tools and prompting genuinely cannot achieve acceptable performance, since multi-agent systems introduce substantial additional surface area for failure and are considerably harder to evaluate and debug than a single-agent pipeline.

Chapter 15: Evaluation and Testing of GenAI Systems

15.1 Why GenAI Evaluation Is Different

Traditional software testing checks a system against deterministic expected outputs. GenAI systems produce open-ended, often non-deterministic output, and "correctness" is frequently a matter of degree and judgment rather than a binary match. This does not mean evaluation is impossible — it means it requires different tools: structured rubrics, model-based judges, and statistical analysis over representative test sets, rather than simple assert-equals checks.

15.2 Building a Golden Test Set

A well-constructed evaluation set — often called a golden set — is one of the highest-leverage investments a GenAI engineering team can make. Effective golden sets are built from real, representative examples of production traffic (not just hand-picked easy cases), cover known edge cases and previously observed failure modes, include adversarial or unusual examples that stress-test the system's robustness, and are large enough to detect meaningful regressions statistically, while remaining small enough to run frequently during development. Golden sets should be treated as living artifacts, continuously expanded as new failure modes are discovered in production.

15.3 LLM-as-Judge Evaluation

Because human evaluation of open-ended output does not scale to the iteration speed required during development, it has become common to use a capable language model as an automated judge, scoring or comparing outputs against a rubric. This approach scales well and correlates reasonably with human judgment when implemented carefully, but has known failure modes: judges can exhibit position bias (favoring whichever output is presented first), length bias

(favoring longer outputs regardless of quality), and self-preference bias (favoring outputs generated by the same model family as the judge). Mitigations include randomizing presentation order, providing the judge with a detailed and specific rubric rather than a vague quality instruction, and periodically validating the judge's scores against a sample of human ratings to confirm they remain well-calibrated.

15.4 Human Evaluation

Despite the scalability of automated evaluation, human review remains essential, particularly for high-stakes decisions, for validating that automated metrics are actually tracking the qualities that matter, and for surfacing failure modes that automated evaluation was not designed to catch. Effective human evaluation programs use clear rubrics (to improve inter-rater consistency), track inter-rater agreement as a signal of rubric quality, and sample production traffic on an ongoing basis rather than evaluating only at development time.

15.5 Regression Testing and Continuous Evaluation

Because prompts, retrieval configurations, and underlying models change frequently, GenAI systems need continuous evaluation infrastructure analogous to a traditional CI/CD test suite: automated evaluation runs against the golden set on every meaningful change, clear thresholds or statistical tests for what constitutes an acceptable regression, and the ability to compare a proposed change against the current production baseline on identical inputs before shipping. This is particularly important because a change intended to fix one failure mode can silently introduce regressions elsewhere, and without systematic evaluation, such regressions are often only discovered after they have affected real users.

15.6 Key Metrics Beyond Correctness

Comprehensive evaluation extends beyond raw task correctness to include: latency (both average and tail latency, since a slow response can be as damaging to user experience as an incorrect one), cost per interaction, safety metrics (rate of policy-violating or harmful output), consistency (whether the system produces stable answers to semantically equivalent queries), and calibration (whether the system's expressed confidence, where applicable, tracks its actual accuracy).

Chapter 16: LLMOps and Deployment

16.1 From Notebook to Production

A prompt or pipeline that works well in an interactive notebook or playground environment requires substantial additional engineering before it is ready for production: robust error handling for malformed or unexpected model output, retry and fallback logic for API failures or rate limits, request queuing and concurrency control, structured logging sufficient to debug production issues after the fact, and integration with the organization's existing deployment, monitoring, and incident-response tooling.

16.2 API vs. Self-Hosted Deployment

Organizations choosing between consuming a model via a hosted API and self-hosting an open-weight model face a genuine tradeoff. Hosted APIs offer simplicity, automatic access to model improvements, and no infrastructure burden, but carry per-token cost that scales with usage, potential data residency or privacy constraints (since requests leave the organization's infrastructure), and dependency on a third party's availability and rate limits. Self-hosting offers control over data residency, potentially lower marginal cost at high volume, and the ability to customize the serving stack, but requires meaningful infrastructure investment, GPU capacity planning, and in-house expertise to operate reliably and keep pace with the rapid evolution of open-weight models.

16.3 Inference Optimization

Self-hosted deployments in particular benefit from a range of inference optimization techniques: quantization (reducing numerical precision of model weights to reduce memory footprint and increase throughput, at some cost to output quality), key-value cache optimization (reusing the attention computation for previously-seen tokens rather than recomputing it on every new token generated), continuous batching (dynamically grouping multiple concurrent requests to maximize GPU utilization, rather than processing requests strictly one at a time or in fixed-size static batches), and speculative decoding (using a smaller, faster draft model to propose multiple tokens ahead, which the larger target model then verifies in parallel, often significantly increasing generation throughput for the same output quality).

16.4 Caching Strategies

Because model inference is a meaningful cost and latency driver, caching is a high-value optimization. Common caching layers include exact-match response caching (storing and reusing the response for an identical prompt, most useful for high-frequency, low-variability queries), semantic caching (matching new queries against cached queries by semantic similarity rather than exact string match, which broadens cache hit rates at the cost of some risk of returning a subtly mismatched cached response), and prompt-prefix caching (a capability offered by several inference providers that caches the computation for a shared prompt prefix, such as a long system prompt or retrieved context, across multiple requests that share that prefix, substantially reducing both cost and latency for that portion of the prompt).

16.5 Rate Limiting, Backpressure, and Graceful Degradation

Production GenAI systems must handle the reality that upstream model providers impose rate limits, and that demand for a system can spike beyond what its model capacity can handle. Robust systems implement client-side rate limiting and request queuing to stay within provider limits, backpressure mechanisms that degrade gracefully under load (such as falling back to a smaller, faster, or cheaper model when the primary model is saturated, or returning a clear "system busy" response rather than an opaque timeout), and circuit breakers that stop sending requests to a failing dependency for a cooldown period rather than compounding an outage with retry storms.

16.6 Observability for GenAI Systems

GenAI-specific observability extends beyond traditional application metrics to include full request/response tracing (capturing the complete prompt sent to the model and the complete response received, which is essential for debugging quality issues after the fact), token usage and cost tracking at a granular level (per request, per feature, per customer), latency breakdowns across pipeline stages (retrieval time versus generation time versus tool-execution time), and quality signals sampled from live traffic (such as periodic automated or human evaluation of a sample of production interactions, and tracking of explicit user feedback signals like thumbs up/down where available).

Chapter 17: Cost Optimization and Scaling

17.1 Understanding the Cost Structure

GenAI system costs are typically dominated by model inference, priced per token for hosted APIs, and are driven by three factors: the number of requests, the number of input tokens per request (which includes system prompts, retrieved context, and conversation history — often the largest contributor), and the number of output tokens generated. Understanding which of these dominates a given system's cost profile is the necessary first step before optimizing, since the right optimization differs substantially depending on whether a system is bottlenecked by high request volume, large input contexts, or long generated outputs.

17.2 Model Selection and Routing

Not every request requires the most capable (and most expensive) available model. Model routing architectures classify incoming requests — by complexity, by task type, or by a lightweight model's own confidence assessment — and route simpler requests to smaller, cheaper, faster models while reserving the most capable models for requests that genuinely require them. This can produce substantial cost savings without a proportional quality reduction, provided the routing logic itself is reliable, since misrouting a complex request to an under-capable model produces a worse outcome than the cost savings justify.

17.3 Reducing Token Consumption

Direct techniques for reducing token consumption include trimming system prompts and retrieved context to only what is genuinely necessary (a common anti-pattern is retrieving and including far more context than the model actually uses), summarizing or truncating long conversation histories rather than including the full history verbatim on every turn, using prompt-prefix caching for stable, reused portions of a prompt, and setting appropriate output length limits or instructing the model explicitly toward concision where verbose output is not needed.

17.4 Batch Processing for Non-Real-Time Workloads

For workloads that do not require real-time responses — bulk document processing, offline analysis, dataset generation — batch inference APIs offered by most major providers process

large volumes of requests asynchronously at a meaningful cost discount compared to real-time API calls, in exchange for higher and less predictable latency (often measured in hours rather than seconds). Identifying which workloads genuinely require real-time latency versus which can tolerate batch processing is a straightforward but frequently overlooked cost optimization.

17.5 Capacity Planning for Self-Hosted Inference

Organizations self-hosting models need to plan GPU capacity against expected load, accounting for the difference between average and peak demand, the latency and throughput characteristics of their chosen model and hardware combination, and the operational cost of maintaining headroom for traffic spikes versus the cost savings of running near full utilization. Autoscaling infrastructure for GPU-backed inference is considerably less mature than for traditional stateless web services, since GPU instances are more expensive to provision on demand and model loading time can introduce meaningful scale-up latency, making capacity planning a more deliberate, forecast-driven exercise than typical web service autoscaling.

Chapter 18: Security and Safety in GenAI Systems

18.1 Prompt Injection

Prompt injection occurs when untrusted content — user input, retrieved documents, tool output, or any other text that enters a model's context — contains instructions designed to override the system's intended behavior. Direct prompt injection involves a user directly attempting to manipulate the system prompt or instructions; indirect prompt injection is more insidious, occurring when the malicious instructions are embedded in content the model retrieves or processes as data, such as a webpage a model is summarizing or a document returned by a search tool, without the user necessarily being aware the injected content is there. Indirect injection is a particularly serious risk for agentic systems with tool access, since a successful injection can result in unauthorized real-world actions, not just an undesired text response.

18.2 Defense Strategies Against Injection

No single technique fully eliminates prompt injection risk, so defense should be layered: clearly delimiting and labeling untrusted content within a prompt (distinguishing system instructions from retrieved or user-supplied data), instructing the model explicitly to treat delimited content as data to be processed rather than instructions to be followed, minimizing the privileges and tool access available to any agent that processes untrusted content, validating and constraining tool call arguments at the application layer regardless of what the model requests, and, for the highest-consequence actions, requiring human approval before execution rather than allowing full autonomy.

18.3 Data Privacy and Leakage

GenAI systems that process sensitive data face several distinct privacy risks: training data or fine-tuning data potentially being reproduced or inferred from model outputs (a documented risk for models fine-tuned on datasets containing sensitive information), sensitive information

from one user's context inadvertently leaking into another user's session in a poorly isolated multi-tenant system, and third-party model providers potentially retaining submitted data for purposes such as abuse monitoring or model improvement, which has direct implications for compliance with data protection regulations and contractual confidentiality obligations. Organizations handling sensitive data should carefully review a model provider's data retention and usage policies, and should not assume that "the data isn't stored" is true without explicit contractual and technical verification.

18.4 Content Moderation and Output Filtering

Production-facing GenAI systems typically require moderation of both user input and model output, to prevent the system from being used to generate harmful content and to catch cases where a model produces unsafe output despite alignment training. Practical moderation architectures often combine a dedicated classification model (trained specifically to detect categories of harmful content) as a fast, cheap first pass, with the primary generative model's own built-in safety training as a second layer, and human review processes for ambiguous or high-risk cases that automated systems flag but cannot confidently resolve.

18.5 Denial of Service and Abuse Vectors

GenAI systems face abuse patterns distinct from traditional web services: adversarial inputs designed to maximize token consumption or trigger expensive retrieval or tool-calling chains, automated scraping of a system's outputs to extract training-quality data or replicate its behavior, and attempts to use a system's tool access as a proxy for unauthorized actions against other systems. Rate limiting, authentication, anomaly detection on usage patterns, and careful scoping of any credentials or permissions available to the system are the primary defenses.

Chapter 19: Responsible AI and Governance

19.1 Why Governance Matters for GenAI Specifically

Generative AI systems introduce governance challenges beyond those of traditional software: outputs are probabilistic and can vary across otherwise similar inputs, failure modes can be subtle and hard to detect through traditional testing, and the potential for harm — misinformation, biased outputs, privacy violations, or unsafe agentic actions — is often less visible at the point of failure than a traditional software bug would be. Organizations deploying GenAI systems at scale increasingly formalize governance processes specifically adapted to these characteristics, rather than relying solely on traditional software governance frameworks.

19.2 Bias and Fairness

Language models trained on large, largely uncurated internet-scale corpora can reproduce and sometimes amplify societal biases present in that data, manifesting in outputs that vary in quality, tone, or accuracy across different demographic groups or subject areas. Addressing this requires deliberate evaluation across relevant subgroups (rather than assuming aggregate quality metrics capture subgroup-level performance), awareness that alignment and safety

training can itself introduce its own biases or blind spots, and, where a system materially affects people's opportunities or access to services, particular caution and likely regulatory obligations around fairness testing and documentation.

19.3 Transparency and Disclosure

Emerging norms and, in a growing number of jurisdictions, legal requirements call for disclosing when a user is interacting with an AI system rather than a human, and for providing meaningful explanation of how AI-driven decisions that affect a person are made, particularly in domains like lending, hiring, and healthcare. Building this transparency in from the start — clear disclosure, explainable decision logic where feasible, and accessible channels for users to contest or seek human review of an AI-driven decision — is considerably easier than retrofitting it after a system is already in production.

19.4 Model Cards and Documentation

Comprehensive documentation of a deployed model or system — often formalized as a "model card" — typically covers the model's intended use cases and explicitly out-of-scope uses, known limitations and failure modes, the data used for training or fine-tuning (to the extent known and disclosable), evaluation results including subgroup breakdowns where relevant, and the version history of changes to the model or system over time. This documentation serves both internal governance purposes and, increasingly, external regulatory and customer disclosure requirements.

19.5 Regulatory Landscape

The regulatory environment for AI systems continues to evolve rapidly and varies significantly by jurisdiction, with some regions adopting risk-tiered frameworks that impose escalating obligations based on a system's potential for harm, and others relying more heavily on existing sector-specific regulation (financial services, healthcare, employment law) applied to AI-specific contexts. GenAI engineers should not treat regulatory compliance as purely a legal team responsibility disconnected from system design — decisions made at the architecture level, such as what data is logged, how model decisions can be audited, and what human oversight mechanisms exist, directly determine whether a system can practically satisfy compliance obligations, and are far cheaper to build in from the start than to retrofit.

Chapter 20: Career Path and Closing Notes

20.1 Building a GenAI Engineering Skill Set

The skills that compound most reliably for a GenAI engineer include a solid foundation in software engineering fundamentals (the majority of the work is still building and operating reliable systems, not model research), a working understanding of the theoretical foundations covered in the early chapters of this handbook (enough to reason about why a system behaves the way it does, not necessarily enough to train a foundation model from scratch), hands-on experience building and iterating on real systems rather than only theoretical knowledge, and

increasingly, judgment about evaluation — knowing how to determine whether a system is actually working well, which is a genuinely difficult and undervalued skill in this field.

20.2 Common Career Entry Points

Engineers arrive at GenAI engineering roles from a range of backgrounds: backend and platform engineers who pick up the model-specific layer of the stack, traditional machine learning engineers who shift focus from training models to building systems around pretrained models, and product-focused engineers who become GenAI specialists through building AI-powered features. There is no single required path, and the field's relative youth means that demonstrated hands-on experience with real systems often weighs as heavily as formal credentials in hiring decisions.

20.3 Staying Current in a Fast-Moving Field

The GenAI field moves quickly enough that specific tools, model capabilities, and best practices in this handbook will inevitably age; the underlying principles — how transformers work, why evaluation is hard, why retrieval and grounding matter, why agentic systems need bounding and oversight — are considerably more durable and are the appropriate foundation to build ongoing learning on top of. Following primary sources (model provider documentation and research publications, rather than only secondary commentary), building small real projects to test new capabilities firsthand, and maintaining a healthy skepticism toward hype in either direction — both overstated capability claims and overstated doom — will serve a GenAI engineer well over a multi-year career in this field.

20.4 Closing Note

This handbook is a foundation, not a finished education. The discipline of GenAI engineering is still actively being defined, and the practitioners who will shape its best practices over the coming years are, in large part, the ones building real systems today and honestly confronting what does and doesn't work. Treat every chapter here as a starting hypothesis to be tested against your own systems, not a fixed set of rules.